

Notice administrateur

Données et outils IA

INSTALLATION OPEN WEBUI ET OUTILS ASSOCIÉS

Deux environnements sont documentés dans ce fichier, avec la même structure pour chaque composant (« Environnement prototype » puis « Environnement production ») :

- **Prototype** : poste de travail de Charlotte (Windows + Docker Desktop), une seule utilisatrice, aucune contrainte de disponibilité.
- **Production (Merlin)** : serveur interne mutualisé, plusieurs chargés d'études, disponibilité continue attendue. Vue d'ensemble de qui tourne où : [docs/architecture/schema-deploiement-prod.md](#).

Sauf mention contraire, une sous-section "Environnement production" ne réexplique pas ce qui est identique au prototype : elle suppose la sous-section prototype déjà lue, et ne documente que ce qui change.

INSTALLATION D'OPEN WEBUI

Environnement prototype (poste local, Windows)

Framework retenu : voir [docs/architecture/decisions.md](#). Ce guide décrit le premier déploiement, en local sur un poste Windows, avant l'hébergement partagé sur Merlin (ci-dessous).

Prérequis

- **Docker Desktop** installé et lancé. Téléchargement : <https://www.docker.com/products/docker-desktop/>
 - À l'installation, choisir le backend **WSL2** si proposé (recommandé sur Windows).
 - Après installation, un redémarrage du poste est parfois demandé.
- Une connexion internet (pour télécharger l'image Open WebUI la première fois).

Étape 1 — Vérifier que Docker fonctionne

Ouvrir un terminal (PowerShell, ou Git Bash intégré à VS Code — les commandes `docker` sont identiques dans les deux) et taper :

```
docker --version # vérif n° de version
docker ps # liste application docker (docker doit être lancé)
```

Étape 2 — Installer et lancer Open WebUI Dans le même terminal :

```
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:main
```

Détail de la commande : - `-d` : lance le conteneur en arrière-plan. - `-p 3000:8080` : rend l'interface accessible sur `http://localhost:3000` (8080 = port dans le conteneur Docker). - `-v open-webui:/app/backend/data` : les données (comptes, conversations, config) sont conservées même si le conteneur est recréé. - `--restart always` : Open WebUI redémarre automatiquement avec Docker Desktop.

Le premier lancement télécharge l'image (plusieurs centaines de Mo, peut prendre quelques minutes).

Étape 3 — Créer le compte admin

Ouvrir `http://localhost:3000` dans un navigateur. Le premier compte créé devient automatiquement administrateur. Choisir un email/mot de passe (peuvent être différents de ceux du poste).

Étape 4 — Vérifier la version

Le fichier JSON disponible à l'adresse `http://localhost:3000/api/config` indique dans les premiers champs la version d'Open WebUI installée.

Le numéro de version est aussi indiqué dans le menu "À propos" du panneau "Réglages", accessible en cliquant sur la bulle "Profil" (en haut à droite ou en bas à gauche).

Étape 5 — Lancer Open WebUI une fois installé

```
docker start open-webui # open webui lancé en arrière-plan
```

Environnement production (Merlin, instance mutualisée)

Delta par rapport au prototype ci-dessus — mêmes principes (image Docker officielle, volume persistant), sur un serveur Linux partagé plutôt qu'un poste Windows individuel.

- **Commande Docker** : même logique que l'étape 2 du prototype :

```
docker run -d -p 1111:0000 -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:main # 1111:0000 = port docker et conteneur à renseigner
```

- **Accès réseau** : `http://srv-gitlab.audiar.net:0000`
- **Volume** : même logique qu'en local (`-v open-webui:/app/backend/data`), sur le disque du serveur Merlin.

Remarque : les applications tournant en docker sur Merlin actuellement sont lancées via la commande `docker compose` et des fichiers de configuration YAML `docker-compose.yml` et `docker-compose.dev.yml`.

Dépannage courant

- **Mettre à jour Open WebUI** :

```
docker pull ghcr.io/open-webui/open-webui:main
docker stop open-webui && docker rm open-webui
```

puis relancer la commande de l'étape 2 (les données sont conservées grâce au volume `open-webui`).

AJOUT DE FOURNISSEURS IA ET PARAMÉTRAGE DES MODÈLES

Les fournisseurs de modèles LLM par API — Mistral, RAGaRenn, OVHcloud AI Endpoints, catalogue comparé dans [docs/benchmark-modeles.md](#) — se connectent via le protocole **OpenAI-compatible**, nativement supporté par Open WebUI.

Source : [OpenAI-Compatible / Open WebUI](#).

Configurer une connexion au modèle (API)

1. Panneau d'administration → Réglages → Connexions → + Ajouter une connexion .
2. Renseigner URL et Auth (Bearer = clé API) du fournisseur.
3. Open WebUI vérifie la connexion en appelant l'endpoint /models du fournisseur avec le Bearer token.
4. Si besoin, préciser les id des modèles visés.
5. Pour faciliter la gestion des modèles une fois les connexions configurées, il est possible de renseigner un ID de préfixe.

Remarque : À l'étape 3, si l'appel à l'endpoint échoue (403/400/401) sans que le fournisseur soit incompatible pour autant, ajouter manuellement les IDs de modèles dans **“Model IDs (Filter)”** de la connexion peut résoudre les problèmes de connexion (la connexion affiche un statut d'erreur mais le chat fonctionne).

Paramétrage des modèles

Une fois les connexions configurées, les modèles disponibles peuvent être gérés dans le menu “Modèles”.

Sources : - [Open WebUI](#) - [Open WebUI](#)

1. Panneau d'administration → Réglages → Modèles .
2. Indiquer quels modèles sont disponibles en activant/désactivant les boutons
3. Panneau d'administration → Réglages → Modèles →  .
4. La configuration du modèle peut être adaptée si besoin :
 - Réglages avancés : permet de renseigner un certain nombre de paramètres des requêtes (nombre maximum de token, paramètres d'échantillonnage, efforts de raisonnement...)
 - Capacités : permet de fixer ce que le modèle a le droit de faire ;
 - Fonctionnalités par défaut : si cochées, elles sont automatiquement activées à chaque nouvelle conversation. Les trois fonctionnalités – recherche web, génération d'images et interpréteur de code – doivent être configurées ailleurs ;
 - Outils intégrés : outils que le modèle peut appeler lui-même via des appels de fonction.
5. Pour rendre les modèles accessibles à d'autres utilisateurs, deux possibilités :
 - configurer l'accès comme “public” ;
 - configurer l'accès comme “privé” et accorder des droits à des utilisateurs ou des groupes.

Comptage des tokens dans l'Analytique

Symptôme : **Panneau d'administration** → **Analytique** affiche **0 token** pour toutes les conversations, alors que les messages et les utilisateurs sont bien comptés.

Cause : Open WebUI ne compte que ce que le fournisseur renvoie dans le champ **usage** de sa réponse, et ce champ est absent des réponses en **streaming** (mode d'affichage par défaut) sauf demande explicite via le paramètre **stream_options** . Vérifié le 30/07/2026 par appel direct à la connexion RAGaRenn (backend Ollama, cf. **system_fingerprint: fp_ollama** dans les réponses) :

Requête envoyée au fournisseur	Champ usage dans la réponse
"stream": false	Oui
"stream": true	Non
"stream": true + "stream_options": {"include_usage": true}	Oui (chunk supplémentaire avant data: [DONE])

Le fournisseur sait donc fournir l'information : il faut la lui demander.

Correctif : cocher la capacité **Usage** du modèle — libellée **Utilisation** en français (confirmé sur l'instance Audiar le 30/07/2026) — qui fait ajouter **stream_options** par Open WebUI sans avoir à désactiver le streaming. - Par modèle : **Panneau d'administration** → **Réglages** → **Modèles** →  → **Capacités** . - Pour tous les modèles : **Panneau d'administration** → **Réglages** → **Modèles** → **Réglages** (bouton en haut à droite) → **Valeurs par défaut** → **Capacités du modèle** → section **Capacités** .

Attention : les réglages de **Valeurs par défaut** peuvent être surchargés par la configuration propre à chaque modèle — vérifier les deux. Le comptage n'est pas rétroactif : tester sur une **nouvelle** conversation, les anciennes restent à 0.

Source : [Langfuse / Open WebUI](#) — « Capture usage (token counts) for OpenAi models while streaming is enabled, you have to navigate to the model settings in Open WebUI and check the "Usage" box below Capabilities. » (À confirmer sur l'instance Audiar : non encore testé au 30/07/2026. Repli si la capacité **Usage** est absente de cette version : **Réglages du modèle** → **Streamer la réponse de la conversation** → **Désactivé** , au prix de la perte de l'affichage progressif des réponses.)

Limite à connaître : même corrigé, ce compteur reste un **agrégat** par modèle et par utilisateur sur une période, sans détail par conversation ni export. Il répond à « combien consomme-t-on au total, avec quel modèle », pas à « quel modèle est le plus économique pour telle tâche » (les prompts diffèrent d'une conversation à l'autre, donc les totaux ne sont pas comparables entre modèles). Une comparaison coût/performance suppose un jeu de prompts figé rejoué à l'identique sur chaque modèle — voir [docs/benchmark-modeles.md](#) .

GESTION DES DROITS

La gestion des droits des utilisateurs et des groupes se gèrent à trois niveaux : - via **Panneau d'administration** → **Réglages** : gestion des accès des utilisateurs et des groupes aux modèles (voir plus haut la section "Paramétrage des modèles") et aux outils externes (serveurs OpenAPI ou serveurs MCP via proxy mcpo) configurés par les administrateurs ; - via **Panneau d'administration** → **Utilisateurs** → **Groupes** : gestion des autorisations de tous les utilisateurs (Autorisations par défaut) et de celles des groupes (**Groupes** dans les réglages du groupe) ; - via **Espaces de travail** : chaque outil, skill, prompt, connaissance et modèle (dans cet espace, un "modèle" associe un modèle d'IA et et un ensemble d'éléments : outils, skills, prompts, connaissances) : gestion de l'accès des utilisateurs et des groupes à ces éléments.

Rôles disponibles

Source officielle : [Roles / Open WebUI](#).

Trois rôles système, aussi utilisés comme valeurs de la colonne `Role` du CSV d'import (ci-dessous) :

Roles	Key	Description
Administrateur	<code>admin</code>	Accès total : gestion des utilisateurs, groupes, config globale
Utilisateur	<code>user</code>	Soumis aux permissions RBAC (défauts + groupes), aucun accès implicite
En attente	<code>pending</code>	Aucun accès tant qu'un admin ne l'a pas approuvé manuellement – recommandé comme rôle par défaut sur une instance partagée (<code>DEFAULT_USER_ROLE=pending</code>)

Premier compte admin

La documentation officielle précise que le tout premier compte créé sur l'instance ("primary administrator") a une protection : pas de bouton de suppression dans l'interface. Elle présente ça comme "un garde-fou de confort, pas une frontière de sécurité". Selon le code source (analyse IA, Claude Sonnet 5), le rôle du primary administrator est verrouillé sur `admin` de façon permanente, au niveau backend (403 `ACTION_PROHIBITED`) : il ne peut pas s'auto-rétrograder et aucun autre admin ne peut changer son rôle.

Les autres comptes admin (créés après le premier) n'ont pas cette protection : leur rôle reste librement modifiable par n'importe quel admin, y compris vers `user` ou `pending` .

Gestion des utilisateurs et des groupes

Deux possibilités pour ajouter des utilisateurs : manuellement ou via un fichier csv, selon un modèle fixe (`docs/user-import.example.csv`).

Concernant la création via un fichier csv, il faut noter qu'il ne permet pas une gestion globale : - si un utilisateur avec la même adresse email existe déjà, il n'est pas ajouté, et la configuration de l'utilisateur déjà enregistré ne change pas ; - si un utilisateur existant n'est pas renseigné dans le csv, son compte n'est pas supprimé.

Ce fichier doit par ailleurs faire l'objet d'une grande vigilance puisqu'il contient les mots de passe des utilisateurs.

Les autorisations des utilisateurs se gèrent via les groupes. Des utilisateurs peuvent être ajoutés seulement une fois le groupe créé.

OUTILS ASSOCIÉS

Comme les modèles, les outils configurés par les administrateurs sont "privés" par défaut.

Plusieurs types d'outils : - serveurs d'outils externes : <https://docs.openwebui.com/features/extensibility/plugin/tools/openapi-servers/>

Toggle global : *Admin Panel* → *Settings* → *Connections* → "*Direct Connections*" → ON nécessaire pour qu'un utilisateur puisse installer ces propres outils

- espace de travail : outil, skill, prompt, connaissance

deux réglages distincts nécessaires côté admin, tous deux désactivés par défaut — 1. (sans lui, personne — même un admin — n'a accès aux connexions directes). En français : *Panneau d'administration* → *Réglages* → *Connexions* → « *Direct connexions* ». 2. **Permission par utilisateur/groupe** : *Admin Panel* → *Users* → *Groups* → *Default permissions (ou un groupe dédié)* → *Features* → “*Direct Tool Servers*” → ON. En français : *Panneau d'administration* → *Utilisateurs* → *Groupe*s → « *Modifier les autorisations par défaut* » → section « *Autorisations des fonctionnalités* » → *toggle* « *Serveur d'outils directs* » (confirmé sur l'instance de Charlotte le 20/07/2026 — désactivé par défaut).

	Admin (Global) – “External Tool Servers”	Utilisateur (Direct) – “Gérer les serveurs d'outils”
Configuré par	L'admin, une seule fois	Chaque utilisateur, individuellement
Visible par	Selon les règles d'accès (contrôle d'accès du serveur : public/privé/groupes)	Seulement l'utilisateur qui l'a ajouté
D'où part l'appel réseau	Le backend/conteneur Open WebUI (localhost = le conteneur)	Le navigateur de l'utilisateur (localhost = son propre poste)

Gestion des secrets (.env) en environnement mutualisé

Transverse à tous les serveurs MCP (pas seulement mcp-postgres ou mcp-qgis) — écrit une fois ici, référencé depuis chaque section serveur plutôt que réexpliqué à chaque fois.

- **Prototype (poste de Charlotte)** : un seul .env local à la racine du repo, jamais commité (voir .gitignore), rempli à partir de config/env.example . Charlotte est seule à le détenir et à le modifier.
- **Production (Merlin)** : le .env réel change de nature selon le type de serveur MCP concerné —
 - **Serveurs centralisés** (mcp-postgres) : un seul .env partagé, sur le serveur Merlin lui-même, aux côtés du conteneur Open WebUI. Question ouverte, pas encore tranchée : qui, côté équipe, a le droit de le modifier (rotation de DATABASE_URI , des clés MCPO_API_KEY_*) et comment il est protégé en accès (droits fichiers, coffre-fort de secrets type Vault/Docker secrets) — cf. docs/architecture/decisions.md , section “Point de vigilance : passage à l'échelle”.
 - **Serveurs liés à un poste individuel** (mcp-qgis) : chaque chargé d'études garde son propre .env local sur son poste, comme Charlotte aujourd'hui — pas de fichier partagé, mais autant de .env que de postes à maintenir en cohérence (même variable MCPO_API_KEY_QGIS , valeur potentiellement différente par poste). Packaging à industrialiser — voir “Checklist de déploiement” ci-dessous.
- Dans tous les cas : ne jamais committer un .env réel (seul .env.example l'est), et faire correspondre chaque clé MCPO_API_KEY_* entre le fichier et la configuration Bearer côté Open WebUI.

CONFIGURER LES SERVEURS MCP (PRINCIPE GÉNÉRAL)

Plusieurs serveurs MCP métier doivent être installés : QGIS, PostgreSQL, filesystem, Excel en particulier.

Principe général

Chaque serveur MCP est exposé via mcpo (proxy MCP → OpenAPI, open-webui/mcpo), puis ajouté dans Open WebUI comme connexion **OpenAPI** (pas MCP natif). Raison de ce choix, valable pour tous les serveurs métier du projet : voir docs/architecture/decisions.md , section “Connexion des serveurs MCP à Open WebUI : pourquoi mcpo ”.

Où configurer les serveurs

Deux écrans distincts, avec un mécanisme différent — vérifié le 22/07/2026 (détail : docs/architecture/decisions.md , section “Global (Admin) vs Direct (personnel)”):

- **Admin Panel → Settings → Outils → External Tool Servers** (“Global”) : configuré une fois par l'admin, visible selon les règles d'accès. L'appel part du **backend/conteneur** Open WebUI — localhost y désigne le conteneur lui-même, jamais le poste de l'utilisateur qui discute.
- **Réglages personnels** (icône de profil, en haut à droite) → **Intégrations → “Gérer les serveurs d'outils”** (“Direct”) : configuré individuellement par chaque utilisateur. L'appel part du **navigateur** de cet utilisateur — localhost y désigne son propre poste, quel que soit l'endroit où tourne le backend Open WebUI.

Le choix entre les deux dépend de la cible, pas seulement de l'endroit où tourne l'outil : une cible unique valable pour tout le monde peut passer par Global (ex. un serveur centralisé à identité partagée) ; une cible propre à chaque utilisateur doit passer par Direct, même si l'outil tourne physiquement sur Merlin (ex. mcp-qgis , ou tout serveur avec des identifiants personnels).

Dans les deux cas, un bouton “+” permet d'ajouter une connexion. Type à choisir : **OpenAPI**, avec l'URL de mcpo et une clé Bearer.

Astuce Docker : si le fournisseur tourne sur l'hôte (pas le cas de Mistral/RAGaRenn/OVH, tous distants), remplacer localhost par host.docker.internal dans l'URL — même logique que pour les serveurs MCP (cf. ci-dessus).

CHECKLIST DE DÉPLOIEMENT D'UN NOUVEAU SERVEUR MCP (INSTANCE MUTUALISÉE)

Modèle générique à appliquer avant de proposer un nouveau serveur MCP aux chargés d'études (au-delà de mcp-qgis , déjà appliqué et validé — statut détaillé dans servers/mcp-qgis/README.md) :

1. Packager sur chaque poste le lancement conjoint de la chaîne locale complète (ex. pour QGIS : plugin QGIS MCP démarré automatiquement à l'ouverture de QGIS → mcpo , qui lance lui-même le serveur MCP) — idéalement un script unique, pas plusieurs manipulations manuelles par un profil non-développeur.
2. Vérifier que le navigateur (onglet Open WebUI) peut effectivement atteindre http://localhost:<port> de mcpo sans blocage CORS/mixed-content (Open WebUI en HTTPS appelant un localhost en HTTP peut être bloqué par certains navigateurs — à tester en conditions réelles).
3. Chaque chargé d'études ajoute lui-même l'URL de son mcpo local dans Settings → Integrations → Manage Tool Servers (FR : Réglages → Intégrations → Gérer les serveurs d'outils) — paramètre personnel, pas partageable entre postes.

Statut d'application de cette checklist par serveur : documenté dans le README du serveur concerné (servers/mcp-<nom>/README.md).

MCP-QGIS (PREMIER SERVEUR MCP MIS EN PLACE)

Priorité retenue par Charlotte le 20/07/2026 : QGIS avant les autres serveurs. Décision et montage technique complets : voir servers/mcp-qgis/README.md .

Environnement prototype (poste de Charlotte)

Pas-à-pas concret suivi sur le poste de Charlotte.

PRÉREQUIS

- QGIS Desktop installé (testé avec la version 3.40.15 “Bratislava” — nécessite ≥ 3.28).
- Plugin **QGIS MCP** installé depuis QGIS (Extensions → Gérer et installer les extensions → rechercher “QGIS MCP”) — installé et à jour en v0.7.0.
- **uv** (gestionnaire de paquets/outils Python moderne) installé sur le poste — voir Étape 1.

ÉTAPE 1 — INSTALLER UV

Dans Git Bash :

```
uv --version
```

Si la commande n’est pas reconnue (`bash: uv: command not found`), l’installer via pip :

```
pip install uv
```

VS Code affiche parfois une notification proposant de créer un environnement virtuel Python pour isoler l’installation — pas nécessaire ici, **uv** est un outil en ligne de commande global, pas une dépendance de projet. On peut fermer la notification (“Don’t show again”).

Une fois installé, si `uv --version` ne fonctionne toujours pas immédiatement, fermer et rouvrir le terminal (le PATH n’est pas toujours actualisé dans la session en cours).

Pourquoi `uv` / `uvx` plutôt qu’un `pip install` classique du serveur MCP ? `uvx` (comme `npx` en JavaScript) exécute un outil dans un environnement isolé et mis en cache, sans l’installer “en dur” dans le projet ni dans le Python global du poste : - Pas de `venv` à créer/activer manuellement à chaque fois. - Toujours la version exacte du dépôt GitHub visé (`--from git+https://...`), sans étape d’installation séparée à maintenir. - Le cache est stocké globalement (`%LOCALAPPDATA%\uv\cache` sous Windows), pas dans le dossier du projet — rien ne pollue `dev_outils_dataIA` . - Le téléchargement ne se fait qu’une fois (mise en cache) ; les lancements suivants réutilisent le cache et démarrent vite. Pour forcer une re-vérification de la dernière version sur GitHub, ajouter `--refresh-package qgis-mcp` à la commande (plus lent, dépend du réseau) — pas nécessaire au quotidien.

C’est la méthode d’installation documentée par le projet `qgis-mcp` lui-même.

ÉTAPE 2 — CONFIGURER ET DÉMARRER LE PLUGIN CÔTÉ QGIS

Dans QGIS, ouvrir le panneau **QGIS MCP** et cliquer sur l’icône de configuration (“**Setup & Configurator**”). Deux cases à cocher pertinentes (ignorer le dropdown “Client” et le bouton “Apply Config” — prévus pour des clients CLI comme Claude Code/Cursor, pas pour Open WebUI) :

- **“Start MCP server automatically when QGIS opens”** → cocher. Démarre automatiquement le socket du plugin à chaque ouverture de QGIS (évite de cliquer “Start Server” manuellement à chaque session).
- **“Always pull latest server from GitHub”** → laisser décoché. Équivalent de l’option `--refresh-package` : forcerait une revérification GitHub à chaque lancement (plus lent). À cocher seulement si on veut explicitement forcer une mise à jour.

Fermer cette fenêtre, puis dans le panneau QGIS MCP principal, cliquer sur **“Start Server”** pour démarrer le socket dès maintenant (le démarrage automatique ne s’appliquera qu’au prochain lancement de QGIS).

ÉTAPE 3 — LANCER LE SERVEUR MCP VIA MCPO

`qgis-mcp-server` est exposé via `mcpo` (proxy MCP → OpenAPI, [open-webui/mcpo](#)) plutôt qu'en connexion MCP native directe — raison détaillée dans [docs/architecture/decisions.md](#) (section "Connexion des serveurs MCP à Open WebUI : pourquoi `mcpo`"). `mcpo` lance lui-même `qgis-mcp-server` en sous-processus (transport `stdio`) et expose une API OpenAPI classique.

Créer un fichier `.env` à la racine du repo (non versionné, voir `.gitignore`) à partir de `config/env.example`, en renseignant au moins `MCPO_API_KEY_QGIS` (la clé à réutiliser côté Open WebUI, authentification Bearer) — cf. "Gestion des secrets (.env)" ci-dessus. Puis, dans Git Bash, lancer le script packagé plutôt que la commande brute :

```
./servers/mcp-qgis/start.sh
```

`start.sh` charge automatiquement ce `.env` (variables exportées via `set -a / source / set +a`) avant de lancer `uvx mcpo --port 8001 --api-key "$MCPO_API_KEY_QGIS" -- uvx --from git+https://github.com/nkarasiak/qgis-mcp qgis-mcp-server --port 8001` choisi arbitrairement, tout ce qui suit `--` étant la commande du serveur MCP que `mcpo` doit lancer lui-même.

Laisser ce terminal ouvert tant que la connexion QGIS doit rester disponible dans Open WebUI (cf. point de vigilance sur la supervision des processus à l'échelle, [docs/architecture/decisions.md](#)).

ÉTAPE 4 — AJOUTER LA CONNEXION DANS OPEN WEBUI

`mcp-qgis` a une cible propre à chaque utilisateur (le poste de chacun) : c'est donc toujours le chemin **Direct** (Réglages personnels → Intégrations → Gérer les serveurs d'outils, cf. "Où configurer les serveurs" ci-dessus), jamais Global. Valeurs :

- **URL** : `http://localhost:8001` (jamais `host.docker.internal` sur ce chemin — vérifié le 22/07/2026, cf. [docs/architecture/decisions.md](#))
- **Auth** : `Bearer`, avec la même clé que `--api-key`

Enregistrer, puis tester dans une conversation (activer l'outil, demander un `ping`).

Débannage : `ImportError: cannot import name 'streamablehttp_client'`

Symptôme au lancement de `start.sh / start.ps1` :

```
ImportError: cannot import name 'streamablehttp_client' from
'mcp.client.streamable_http'
```

Cause : `mcpo` déclare sa dépendance à `mcp` sans plafond de version (`mcp>=1.17.0`, aucune borne haute). Depuis la sortie de `mcp` 2.0.0, la fonction importée par `mcpo` a été renommée (`streamablehttp_client` → `streamable_http_client`), cassant la compatibilité. `uv` résolvant par défaut la version la plus récente disponible, toute installation (même sur un poste neuf, sans lien avec un cache local) récupère `mcp` 2.0.0 et échoue.

Correctif appliqué dans `start.sh / start.ps1` : figer explicitement les deux versions testées comme compatibles ensemble, pas seulement celle de `mcpo` :

```
uvx --with "mcp==1.29.0" mcpo@0.0.20 --port 8001 --api-key "..." -- ...
```

À revoir si une future version de `mcpo` déclare officiellement un support de `mcp` 2.x — retester avant de lever ce pin.

Environnement production (déploiement à plusieurs postes)

`mcp-qgis` reste un serveur **par poste** en production, jamais centralisé (le plugin agit sur le projet QGIS ouvert à l'écran de chaque agent — cf. `docs/architecture/schema-deploiement-prod.md`). Le delta par rapport au prototype ci-dessus est donc surtout un delta de **packaging**, pas d'architecture :

- Automatiser la chaîne complète (plugin QGIS → `qgis-mcp-server` → `mcpo`) au démarrage du poste, sans terminal laissé ouvert manuellement — cf. checklist générique ci-dessus et `docs/architecture/decisions.md`, section “passage à l'échelle” (piste non tranchée : tâche planifiée Windows ou service via NSSM/WinSW).
- Chaque agent garde son propre `.env` local (`MCPO_API_KEY_QGIS`) — cf. “Gestion des secrets” ci-dessus.
- Chaque agent ajoute lui-même son `mcpo` local dans ses réglages personnels Open WebUI (*Direct Tool Servers*, cf. checklist ci-dessus) — pas une configuration partagée entre postes.
- **CORS à vérifier une fois Open WebUI sur Merlin** : le navigateur enverra l'origine de Merlin (pas `localhost`) dans sa requête vers le `mcpo` local de chaque poste — `mcpo` / `qgis-mcp-server` doivent l'accepter en CORS, sans quoi la connexion échoue malgré des identifiants corrects. Non testé au-delà d'un Open WebUI servi en local (origine triviale).

Statut détaillé de ce qui est déjà validé vs. restant à faire : `servers/mcp-qgis/README.md`, section “Checklist de déploiement”.

MCP-POSTGRES

Serveur MCP pour interroger la base de données PostgreSQL métier (cas d'usage 2 de l'analyse fonctionnelle). Décision d'implémentation et comparatif : `servers/mcp-postgres/README.md` et `docs/architecture/benchmark-techno.md`.

Environnement prototype (poste de Charlotte)

PRÉREQUIS

- `uv` / `uvx` déjà installé (fait lors du montage `mcp-qgis`, voir ci-dessus).
- Identifiants de connexion à une base réelle (host, port, nom de la base, utilisateur, mot de passe) — idéalement un rôle PostgreSQL dédié en lecture seule (`GRANT SELECT` uniquement), pas le compte administrateur. Procédure de création/vérification du rôle : `servers/mcp-postgres/README.md`, section “Vérification du rôle lecture seule”.
- Accessibilité réseau depuis le poste de Charlotte (VPN, pare-feu) vers le serveur PostgreSQL cible, à confirmer.

ÉTAPE 1 — CONFIGURER LE `.ENV`

Compléter le `.env` local (cf. “Gestion des secrets (.env)” ci-dessus) à partir de `config/env.example`, avec `DATABASE_URI` (idéalement le rôle lecture seule) et `MCPO_API_KEY_POSTGRES`.

ÉTAPE 2 — LANCER LE SERVEUR MCP VIA `MCPO`

```
./servers/mcp-postgres/start.sh
```

Lance `uvx mcpo --port 8002 --api-key "$MCPO_API_KEY_POSTGRES" -- uvx postgres-mcp --access-mode=restricted` (port `8002`, distinct du `8001` de `mcp-qgis` pour permettre aux deux de tourner en même temps ; mode `--access-mode=restricted` = lecture seule, retenu par défaut — détail complet : `servers/mcp-postgres/README.md`).

ÉTAPE 3 — AJOUTER LA CONNEXION DANS OPEN WEBUI

Chemin **Admin/Global** a priori adapté ici (cible unique partagée, cf. “Où configurer les serveurs” ci-dessus) — à confirmer une fois le choix ci-dessous (Environnement production) réellement tranché :

- **URL** : `http://host.docker.internal:8002` (le conteneur Open WebUI atteint l’hôte — correct sur le chemin Global, jamais sur le chemin Direct)
- **Auth** : Bearer , avec la même clé que `--api-key`

Si les identifiants PostgreSQL personnels par utilisateur sont finalement retenus (cf. “Environnement production” ci-dessous), ce test devra être repris via le chemin **Direct** — `http://localhost:8002` depuis les réglages personnels de chaque utilisateur, jamais `host.docker.internal` .

Environnement production (Merlin, centralisé)

À la différence de `mcp-qgis` , `mcp-postgres` n’a aucune raison de dépendre du poste d’un agent en particulier — il interroge une base distante, pas la session QGIS d’un poste précis. Décision déjà prise dans `docs/architecture/schema-deploiement-prod.md` : ce serveur tourne **centralisé sur le même serveur que l’instance Open WebUI mutualisée** (Merlin), pas sur les postes agents.

Pas encore tranché (identifié le 21/07/2026, reformulé le 22/07/2026) — la question n’est plus seulement “comment ce service est géré sur Merlin” mais aussi “une instance partagée ou une par utilisateur”, ce qui détermine directement le chemin de connexion (cf. `docs/architecture/decisions.md` , section “Global (Admin) vs Direct (personnel)”):

- **Un compte technique partagé** (un seul `DATABASE_URI` pour tout le monde) → une seule instance `mcpo / postgres-mcp` sur Merlin, connexion **Global** (Admin), comme testé en prototype ci-dessus.
- **Des identifiants PostgreSQL personnels par utilisateur** (approche que Charlotte souhaite conserver) → N instances `mcpo / postgres-mcp` sur Merlin (une par utilisateur, chacune avec son propre port et son propre `DATABASE_URI`), connectées soit en **Direct** (chaque utilisateur configure lui-même sa propre instance, comme `mcp-qgis`), soit en **Global avec restriction d’accès par connexion** si ce mécanisme se confirme utilisable (cf. bouton “Accès”, non vérifié) — évite de faire reconfigurer chaque utilisateur à la main.

Reste également ouvert, quel que soit le choix ci-dessus : comment ce service est réellement géré sur Merlin (conteneur Docker avec `--restart always` , ou service `systemd`, plutôt qu’une commande manuelle dans un terminal), et qui détient le/les `.env` une fois centralisés (cf. “Gestion des secrets (.env)” ci-dessus — la réponse diffère aussi selon le choix retenu). Ce guide sera complété une fois ces points arbitrés — ne pas improviser une procédure avant.

| MCP-FILESYSTEM

Serveur MCP pour la lecture/modification de fichiers. **Statut : reporté**, pas de priorité dans le scope Open WebUI actuel — aucun des 9 cas d’usage de l’analyse fonctionnelle ne l’appelle (voir `docs/architecture/decisions.md` et `servers/mcp-filesystem/README.md`).

Pas-à-pas à écrire une fois ce serveur configuré à son tour, suivant la même structure “Environnement prototype” / “Environnement production” que `mcp-qgis` et `mcp-postgres` ci-dessus.

RECHERCHE SUR LE WEB

<https://docs.openwebui.com/features/chat-conversations/web-search/providers/external/>

FONCTIONNALITÉS POUR PLUS TARD

Fonctions

Une fonction modifie le comportement du backend Open WebUI lui-même : elle s'exécute côté serveur, indépendamment du function calling du modèle. Il en existe trois types : - Pipe — crée une entrée "modèle" personnalisée qui apparaît dans le sélecteur de modèles, comme si c'était un LLM classique. Sert typiquement à brancher une API externe non supportée nativement, ou à construire un pipeline (RAG custom, agent multi-étapes) qui répond à la place d'un vrai modèle ; - Filter — s'intercale sur le flux inlet (avant que la requête parte vers le modèle) et/ou outlet (après la réponse). Sert à modifier ou enrichir silencieusement les échanges (masquage de données sensibles, injection de contexte, logging, modération...) ; - Action — ajoute un bouton personnalisé sous les messages du chat (ex. "traduire", "reformuler").